

UNIVERSIDADE NOVE DE JULHO – UNINOVE
DIRETORIA DE INFORMÁTICA



INTELIGÊNCIA ARTIFICIAL E
APRENDIZADO DE MÁQUINAS

Bancos de Dados NoSQL: Uma Análise Abrangente

SÃO PAULO

2025

Inteligência Artificial e Aprendizado de Máquinas

Bancos de Dados NoSQL: Uma Análise Abrangente

VITOR HUGO DA SILVA SANTOS

RA:625102765

Trabalho apresentado ao curso de Pós-Graduação Latu Sensu – Inteligência Artificial e Aprendizado de Máquinas da Universidade Nove de Julho, como parte dos requisitos para a obtenção do Grau de Especialista em Inteligência Artificial e Aprendizado de Máquinas.

Orientadores: **Prof. Dr. Fabio Henrique Pereira**

Unidade: Campus: MM

Curso: Pós-Graduação Latu Sensu
Inteligência Artificial e

Aprendizado de Máquinas

Período: Segundo Semestre - 2025

São Paulo
2025

Sumário

1. Introdução
2. Categorias de Bancos de Dados NoSQL
 - 2.1. Bancos Orientados por Coluna
 - 2.2. Bancos de Grafos
 - 2.3. Bancos Chave-Valor
 - 2.4. Bancos de Documentos
3. Conclusão
4. Referências Bibliográficas

1. Introdução

Com o advento da era digital e o crescimento exponencial do volume de dados gerados, os bancos de dados relacionais tradicionais, embora robustos e amplamente utilizados, começaram a apresentar limitações em cenários que exigem alta escalabilidade, flexibilidade de esquema e disponibilidade contínua. Em resposta a esses desafios, surgiu o movimento NoSQL (Not only SQL), que engloba uma variedade de sistemas de gerenciamento de bancos de dados não relacionais, cada um otimizado para tipos específicos de dados e cargas de trabalho.

Este trabalho tem como objetivo explorar as principais categorias de bancos de dados NoSQL: Bancos Orientados por Coluna, Bancos de Grafos, Bancos Chave-Valor e Bancos de Documentos. Para cada categoria, serão apresentadas suas características fundamentais, exemplos de tecnologias proeminentes e casos de uso práticos em empresas reais, justificando a adequação da escolha do modelo NoSQL para o cenário em questão. A análise visa fornecer uma compreensão abrangente de como esses bancos de dados não relacionais estão sendo empregados para solucionar problemas complexos no mundo da tecnologia da informação.

2.1. Bancos Orientados por Coluna

2.1.1. Breve descrição da categoria

Bancos de dados orientados por coluna, também conhecidos como Column-Family Databases, armazenam dados em colunas, em vez de linhas, o que difere fundamentalmente dos bancos de dados relacionais. Essa organização otimiza o desempenho para consultas analíticas que acessam apenas um subconjunto de colunas. Ao invés de ler linhas inteiras, o sistema pode focar apenas nas colunas relevantes, resultando em uma leitura de dados mais eficiente e rápida para cargas de trabalho de big data e análise em tempo real.

2.1.2. Exemplo de tecnologia: Apache Cassandra

O Apache Cassandra é um sistema de gerenciamento de banco de dados NoSQL distribuído, de código aberto, projetado para lidar com grandes volumes de dados em vários servidores, oferecendo alta disponibilidade e escalabilidade linear sem um único ponto de falha. Sua arquitetura descentralizada permite que os dados sejam distribuídos de forma homogênea entre os nós de um cluster, garantindo redundância e resiliência. O

Cassandra é amplamente utilizado em aplicações que exigem alta taxa de escrita e leitura, e tolerância a falhas.

2.1.3. Exemplo de aplicação real: Netflix

A Netflix, líder global em serviços de streaming, utiliza o Apache Cassandra como um componente crucial de sua infraestrutura de backend. A plataforma emprega o Cassandra para gerenciar e armazenar dados críticos relacionados ao histórico de visualização dos usuários, preferências de conteúdo, dados de conta e outras informações operacionais que demandam alta disponibilidade e escalabilidade. A capacidade de lidar com o volume massivo de dados gerados por milhões de usuários simultaneamente é um requisito fundamental para a Netflix.

2.1.4. Justificativa de adequação

O Apache Cassandra é intrinsecamente adequado para as necessidades da Netflix devido à sua escalabilidade horizontal e alta disponibilidade. A natureza global do serviço da Netflix exige um sistema de banco de dados que possa expandir sua capacidade de armazenamento e processamento de forma linear, adicionando mais nós ao cluster, sem interrupções. A arquitetura masterless do Cassandra garante que não haja um único ponto de falha, o que é vital para um serviço que precisa estar disponível 24 horas por dia, 7 dias por semana, em todo o mundo. Além disso, a capacidade do Cassandra de realizar operações de escrita e leitura de alta velocidade em grandes conjuntos de dados é essencial para registrar e acessar a atividade do usuário em tempo real, permitindo funcionalidades como a continuação de vídeos e recomendações personalizadas.

2.2. Bancos de Grafos

2.2.1. Breve descrição da categoria

Bancos de dados de grafos são sistemas especializados na representação e consulta de dados altamente conectados. Eles utilizam uma estrutura de grafo, composta por nós (entidades), arestas (relacionamentos) e propriedades (atributos de nós e arestas), para modelar e armazenar dados. Diferentemente dos bancos de dados relacionais, que exigem operações de JOIN complexas e custosas para navegar por relacionamentos, os bancos de grafos armazenam as conexões como elementos de primeira classe, o que torna as consultas de travessia de relacionamentos extremamente eficientes.

2.2.2. Exemplo de tecnologia: Neo4j

O Neo4j é o banco de dados de grafos mais proeminente e amplamente adotado no mercado. Ele implementa um modelo de grafo de propriedades nativo, o que significa que tanto os nós quanto as arestas podem ter propriedades. O Neo4j é conhecido por seu alto desempenho em consultas que exploram relacionamentos profundos e complexos em tempo real, sendo uma ferramenta poderosa para casos de uso como sistemas de recomendação, detecção de fraudes e gerenciamento de redes.

2.2.3. Exemplo de aplicação real: Airbnb

O Airbnb, plataforma online de aluguel de acomodações, utiliza o Neo4j para aprimorar seu sistema de recomendações e para mapear as conexões sociais entre seus usuários. Ao modelar as relações entre usuários, anúncios, localizações e amizades como um grafo, o Airbnb consegue oferecer recomendações mais relevantes e personalizadas, como, por exemplo, sugerir acomodações de anfitriões que são amigos de amigos do usuário, o que aumenta a confiança e a probabilidade de reserva.

2.2.4. Justificativa de adequação

O Neo4j é a escolha ideal para o Airbnb porque o modelo de grafos reflete de forma natural e intuitiva a complexa rede de relacionamentos presente em sua plataforma. Consultas que seriam extremamente complexas e lentas em um banco de dados relacional, como "encontrar anúncios em uma cidade específica de anfitriões que são amigos de amigos de um determinado usuário", tornam-se simples e rápidas em um banco de dados de grafos. A capacidade de atravessar o grafo de forma eficiente permite que o Airbnb gere recomendações em tempo real, melhore a experiência do usuário e promova um senso de comunidade e confiança entre seus membros.

2.3. Bancos Chave-Valor

2.3.1. Breve descrição da categoria

Bancos de dados chave-valor representam a forma mais simples de armazenamento de dados NoSQL. Eles operam com base em um modelo de dados que associa uma chave única a um valor, onde o valor pode ser qualquer tipo de dado (string, JSON, imagem, etc.). A simplicidade de sua estrutura permite operações de leitura e escrita extremamente rápidas, pois o acesso aos dados é direto, sem a necessidade de esquemas complexos ou operações de JOIN. Isso os torna ideais para cenários que exigem alta performance e baixa latência para acesso a dados simples e voláteis.

2.3.2. Exemplo de tecnologia: Redis

Redis (Remote Dictionary Server) é um armazenamento de estrutura de dados em memória, de código aberto, que pode ser utilizado como banco de dados, cache e message broker. Sua principal característica é a persistência opcional e a capacidade de armazenar dados diretamente na memória RAM, o que proporciona latência de milissegundos para operações de leitura e escrita. O Redis suporta diversas estruturas de dados, como strings, hashes, lists, sets e sorted sets, tornando-o versátil para uma ampla gama de aplicações, incluindo caching, gerenciamento de sessões, filas de mensagens e leaderboards.

2.3.3. Exemplo de aplicação real: Twitter

O Twitter (atualmente conhecido como X) utiliza o Redis em sua infraestrutura para gerenciar as timelines dos usuários. Quando um usuário publica um tweet, ele precisa ser entregue de forma eficiente para os feeds de milhões de seguidores. O Redis é empregado para armazenar os IDs dos tweets em listas que representam as timelines de cada usuário, permitindo uma atualização e recuperação rápidas.

2.3.4. Justificativa de adequação

O Redis é uma escolha excelente para o caso de uso do Twitter devido à sua velocidade incomparável e à flexibilidade de suas estruturas de dados. A necessidade de gerar e entregar timelines em tempo real para uma vasta base de usuários exige um sistema que possa lidar com um volume massivo de operações de escrita e leitura com latência mínima. Como o Redis opera predominantemente em memória, ele consegue adicionar novos tweets às timelines de milhões de seguidores quase instantaneamente. A utilização de listas no Redis simplifica o processo de gerenciamento das timelines, permitindo que novos tweets sejam facilmente inseridos no topo e que as timelines sejam truncadas para manter um tamanho gerenciável. Além disso, o uso do Redis para caching das timelines reduz significativamente a carga sobre os bancos de dados persistentes, garantindo uma experiência de usuário fluida e responsiva.

2.4. Bancos de Documentos

2.4.1. Breve descrição da categoria

Bancos de dados de documentos são uma categoria de bancos de dados NoSQL que armazenam dados em formatos semiestruturados, geralmente como documentos JSON (JavaScript Object Notation) ou BSON (Binary JSON). Cada documento é uma unidade autônoma que pode conter dados complexos, incluindo arrays e objetos aninhados, e não exige um esquema predefinido. Essa flexibilidade de esquema permite que os desenvolvedores armazenem e recuperem dados de forma mais ágil, adaptando-se facilmente às mudanças nas necessidades da aplicação. Eles são ideais para aplicações que lidam com grandes volumes de dados que não se encaixam bem em um modelo relacional rígido.

2.4.2. Exemplo de tecnologia: MongoDB

O MongoDB é um dos bancos de dados de documentos mais populares e amplamente utilizados. Ele armazena dados em documentos flexíveis, semelhantes a JSON, o que significa que os campos podem variar de documento para documento e a estrutura de dados pode ser alterada ao longo do tempo sem a necessidade de migrações complexas. O MongoDB oferece uma linguagem de consulta rica e poderosa, recursos de indexação robustos e alta escalabilidade, tornando-o adequado para uma vasta gama de aplicações, desde sistemas de gerenciamento de conteúdo até plataformas de e-commerce.

2.4.3. Exemplo de aplicação real: Globo.com

A Globo.com, um dos maiores portais de mídia e entretenimento do Brasil, utiliza o MongoDB em sua infraestrutura para gerenciar o vasto volume de conteúdo digital que produz. Cada artigo de notícia, vídeo, galeria de fotos ou outro tipo de conteúdo pode ser armazenado como um documento JSON/BSON no MongoDB. Essa abordagem permite que a Globo.com lide com a diversidade e a evolução constante de seus formatos de conteúdo de forma eficiente.

2.4.4. Justificativa de adequação

O MongoDB é a escolha ideal para a Globo.com devido à sua flexibilidade de esquema e escalabilidade horizontal. A natureza do conteúdo de mídia é inerentemente variada e em constante evolução; um artigo de notícias possui uma estrutura diferente de um vídeo ou de uma galeria de imagens. O modelo de documento flexível do MongoDB permite que a Globo.com armazene esses diferentes tipos de conteúdo sem a necessidade de um esquema rígido e predefinido, o que agiliza o desenvolvimento e a manutenção.

Além disso, a capacidade do MongoDB de escalar horizontalmente, distribuindo os dados entre múltiplos servidores, é crucial para lidar com os picos de tráfego e o volume massivo de acessos que ocorrem durante grandes eventos de notícias, esportes ou entretenimento, garantindo alta disponibilidade e desempenho para milhões de usuários.

3. Conclusão

Os bancos de dados NoSQL emergiram como uma solução poderosa e flexível para os desafios impostos pela crescente demanda por escalabilidade, alta disponibilidade e agilidade no desenvolvimento de aplicações modernas. Conforme demonstrado, cada categoria – Bancos Orientados por Coluna, Bancos de Grafos, Bancos Chave-Valor e Bancos de Documentos – possui características distintas que os tornam ideais para cenários de uso específicos, onde os bancos de dados relacionais tradicionais poderiam apresentar limitações de desempenho ou flexibilidade.

A escolha do banco de dados NoSQL adequado depende intrinsecamente dos requisitos da aplicação, do volume e da natureza dos dados, e da necessidade de flexibilidade de esquema ou de alta performance em operações específicas. Empresas como Netflix, Airbnb, Twitter e Globo.com são exemplos claros de como a adoção estratégica de soluções NoSQL pode impulsionar a inovação e a eficiência, permitindo o desenvolvimento de sistemas robustos e escaláveis que atendem às demandas do mundo digital contemporâneo. A compreensão das particularidades de cada modelo NoSQL é fundamental para arquitetos e desenvolvedores que buscam construir infraestruturas de dados resilientes e de alto desempenho.

4. Referências Bibliográficas

[1] **DevMedia. NoSQL: Guia Rápido para Iniciantes em Bancos de Dados.** Disponível em: <https://www.devmedia.com.br/introducao-aos-bancos-de-dados-nosql/26044>. Acesso em: 14 ago. 2025.

[2] **Data Science Academy. Como Bancos de Dados (Relacionais e ...) - Data Science Academy.** Disponível em: <https://blog.dsacademy.com.br/como-bancos-de-dados-relacionais-e-nao-relacionais-sao-usados-em-projetos-de-data-science/>.

Acesso em: 14 ago. 2025.

[3] **FEMANET. UM ESTUDO EXPLORATÓRIO ACERCA DE BANCO DE DADOS** Disponível em: <https://cepein.femanet.com.br/BDigital/arqPIBIC/1511420151B623.pdf>.

Acesso em: 14 ago. 2025.

[4] **Repositório AEE. análise de desempenho de banco de dados nosql em um.** Disponível em: http://repositorio.aee.edu.br/bitstream/aee/45/1/TCC2_2017_02_DanielFerreiraRabelo_MarcoViniciusIseckeCandido.pdf. Acesso em: 14 ago. 2025.

[5] **UTFPR. UTILIZANDO A TECNOLOGIA DE BANCO DE DADOS NOSQL.** Disponível em: https://riut.utfpr.edu.br/jspui/bitstream/1/15944/2/PG_COCIC_2015_1_03.pdf.

Acesso em: 14 ago. 2025.

[6] **SBC. NoSQL e Segurança: Um estudo de análise para prevenção de** Disponível em: <https://sol.sbc.org.br/index.php/latinoware/article/download/31579/31382/>.

Acesso em: 14 ago. 2025.

[7] **AWS. O que é um banco de dados NoSQL.** Disponível em: <https://aws.amazon.com/pt/nosql/>. Acesso em: 14 ago. 2025.

[8] **Medium. Os 9 Principais Casos de Uso do Banco de Dados NoSQL.** Disponível em: <https://medium.com/@habbema/os-9-principais-casos-de-uso-do-banco-de-dados-nosql-9ad6277ca8fa>. Acesso em: 14 ago. 2025.

[9] **Microsoft Azure. O que são os bancos de dados NoSQL.** Disponível em: <https://azure.microsoft.com/pt-br/resources/cloudcomputing-dictionary/what-is-nosql-database>. Acesso em: 14 ago. 2025.

[10] **MongoDB. O Que É NoSQL? Bancos De Dados NoSQL Explicados.** Disponível em: <https://www.mongodb.com/ptbr/resources/basics/databases/nosql-explained>. Acesso em: 14 ago. 2025.

[11] **Reddit. Como você decide entre usar bancos de dados SQL e NoSQL.** Disponível em: https://www.reddit.com/r/webdev/comments/1gnc5dg/how_do_you_decide_between_using_sql_and_nosql/?tl=pt-br.

Acesso em: 14 ago. 2025.

[12] **Repositório UCS. Estudo de caso de bancos NOSQL no contexto de big data.** Disponível em: <https://repositorio.ucs.br/xmlui/handle/11338/1551?locale-attribute=es>. Acesso em: 14 ago. 2025.

[13] **Repositório UTFPR. Utilizando a tecnologia de banco de dados NoSQL: um caso prático.** Disponível em: <https://repositorio.utfpr.edu.br/jspui/handle/1/15944>.

Acesso em: 14 ago. 2025.

[14] **Repositório UFSM. Ticketflow: um estudo de caso para uso de BD NoSQL ou relacional.** Disponível em: <https://repositorio.ufsm.br/handle/1/35324>.

Acesso em: 14 ago. 2025.

[15] **Repositório UnB. Um estudo de caso com Dados do Bolsa Família e NoSQL Cassandra.** Disponível em: https://bdm.unb.br/bitstream/10483/19381/1/2017_JorgeLuizAndrade.pdf.

Acesso em: 14 ago. 2025.

[16] **Repositório UnB. Uma proposta de arquitetura NoLAP para um sistema de apoio à decisão acadêmico.** Disponível em: <https://repositorio.unb.br/handle/10482/40556>. Acesso em: 14 ago. 2025.

[17] **USCS. Implementação de um Sistema Multi-plataforma para Gerenciamento de Atividades Complementares em Cursos Superiores utilizando Banco de Dados noSQL.** Disponível em:

http://seer.uscs.edu.br/index.php/revista_informatica_aplicada/article/view/6935.

Acesso em: 14 ago. 2025.

[18] **Core. Um estudo sobre a utilização do banco de dados NoSQL Cassandra em Dados Biológicos.** Disponível em: <https://core.ac.uk/download/pdf/196878088.pdf>.

Acesso em: 14 ago. 2025.

[19] **ResearchGate. NoSQL no desenvolvimento de aplicações Web colaborativas.**

Disponível em:

https://www.researchgate.net/profile/BernadetteLoscio/publication/268201466_NoSQL_no_desenvolvimento_de_aplicacoes_Web_colaborativas/links/576aa72008aef2a864d1ef8no-desenvolvimento-de-aplicacoes-Web-colaborativas.pdf. Acesso em: 14 ago. 2025.

[20] **Google Books. NoSQL: Como armazenar os dados de uma aplicação moderna.**

Disponível em:

<https://books.google.com/books?hl=en&lr=&id=cY8tEAAAQBAJ&oi=fnd&pg=PT20&dq=aplica%C3%A7%C3%B5es+de+bancos+de+dados+nosql&ots=WFXEbmVEwj&>

Acesso em: 14 ago. 2025.

[21] **DCOMP UFSCAR. Abordagem NoSQL–uma real alternativa.** Disponível em:

https://www.dcomp.ufscar.br/verdi/topicosCloud/nosql_artigo.pdf.

Acesso em: 14 ago. 2025.

[22] **Google Books. NoSQL essencial: um guia conciso para o mundo emergente da persistência poliglota.** Disponível em:

<https://books.google.com/books?hl=en&lr=&id=UHCiDwAAQBAJ&oi=fnd&pg=PT9&dq=estudos+de+caso+nosql&ots=gY6nGp4jCO&sig=kB7l8FtppxgZwFZOXDuk8R00>.

Acesso em: 14 ago. 2025.